
AN0027-EC NB-IOT 安全启动介绍_V1.1

EigenCOMM Wireless Microcontroller

产品综述

本文档描述移芯通信 NB-IoT 芯片的安全启动方案。

目录

1.	安全启动概述	3
2.	安全启动步骤	3
3.	安全启动软件流程	4
3.1	ROM 验证 bootloader	4
3.2	Bootloader 验证 system	5
4.	ECDSA 密钥及签名	5
4.1	密钥对的产生	5
4.2	编译时使用密钥对的配置	6
4.2.1	编译时的配置	6
4.2.2	ECSecTools 的配置	7
4.3	Security public key 使用	7
4.3.1	security bootloader public key	7
4.3.2	security system public key	7
4.4	签名生成	7
4.4.1	Bootloader 签名	7
4.4.2	System 签名	8
4.5	远程签名	8
4.6	注意事项	8
5.	安全启动配置	8
6.	Image 签名算法	9
7.	Image 烧录	9
8.	安全启动相关	11
9.	附录	11
9.1	参考文档	11
9.2	术语和解释	12
9.3	安全和警告	13
10.	版本	14
11.	关于我们	15

1. 安全启动概述

安全启动的目的时保证芯片只运行用户指定的程序，芯片每次启动时都会验证从 FLASH 加载的 bootloader image 和 system image 是否是用户指定的。

安全启动中采用 ECDSA 签名算法对 bootloader image 和 system image 镜像进行签名和验证，ECDSA 签名算法使用公钥/密钥对，密钥用于对指定的二进制文件进行签名，公钥用于签名验证。

启动过程采用安全链的方式，具体包括固化在芯片内部的 ROM 启动引导程序来验证 bootloader image 是安全的，bootloader image 来验证 system image 是安全的。

验证过程需要产生两组 ECDSA 密钥对，分别用于 bootloader image 和 system image。验证 bootloader image 的公钥需要烧录在 efuse 中，且保证是一次性烧录，从而和硬件是完全绑定的，其他用户无法不可更改。验证 system image 的公钥需要集成编译至 bootloader image，经过安全验证的 bootloader image 是一体的，其他用户无法更改。私钥需要由用户安全保存，不能泄露，否则其他用户用该私钥进行签名的 image 都可以通过启动时的签名验证，安全启动失效。

2. 安全启动步骤

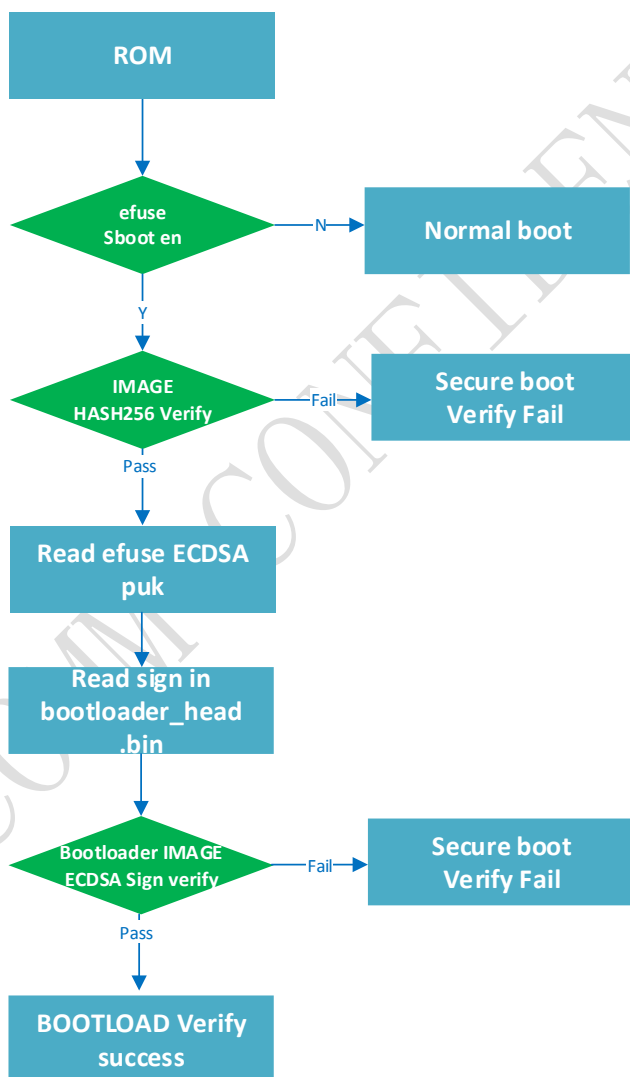
本章描述安全启动概要步骤：

1. 在 SDK 中配置安全启动生效，配置 bootloader 和 system 安全启动使能。
2. 在 SDK ECSecTools 工具中产生和配置 ECDSA 公钥/私钥密钥对文件 securitybl-priv.pem/securitybl-pub.pem 和 securitysys-priv.pem/securitysys-pub.pem，分别用于 bootloader image 和 system image，在 ECSecTools 工具中配置文件路径。
3. 在 SDK 中配置指定安全工具的配置路径，编译时 ECSecTools 工具将 securitybl-pub.pem 中获取公钥 security bootloader pubkey 并集成至 boot loader，ECSecTools 工具也从 securitybl-priv.pem 和 securitysys-priv.pem 中获取私钥以产生对 bootloader image 和 system image 的签名，签名分别保存在 bootloader_head.bin 和 system_head.bin。
4. 用 ECSecTools 将 bootloader.bin/bootloader_head.bin 和 system.bin/system_head.bin 烧录至 FLASH。
5. security bootloader pubkey 烧录
可选方案 1，ECSecTools 烧录 security bootloader pubkey 至 efuse，ECSecTools 工具从 securitysbl-pub.pem 提取 security bootloader pubkey，转换为 efuse_burn.bin，用 ECSecTools 工具烧录至芯片 efuse ecdsa_puk，并烧录 efuse burn_dis_app（禁止烧录标志，防止反复烧录）、sboot_en(安全启动标志)。
可选方案 2，bootloader 烧录 security bootloader pubkey 至 efuse bit，前提在步骤 3 需要配置 bootloader burn security bootloader pubkey efuse bit 使能选项，并将 security bootloader pubkey 嵌入在生成 bootloader_head.bin 中。
6. 若 security bootloader pubkey 烧录选用的是可选方案 2，首次启动时，读取 sboot_en 为未使能，bootloader 将从 bootloader_head.bin 提取 security bootloader pubkey，烧录至 efuse ecdsa_puk，并烧录 efuse burn_dis_app（禁止烧录标志，防止反复烧录）、sboot_en(安全启动标志)。

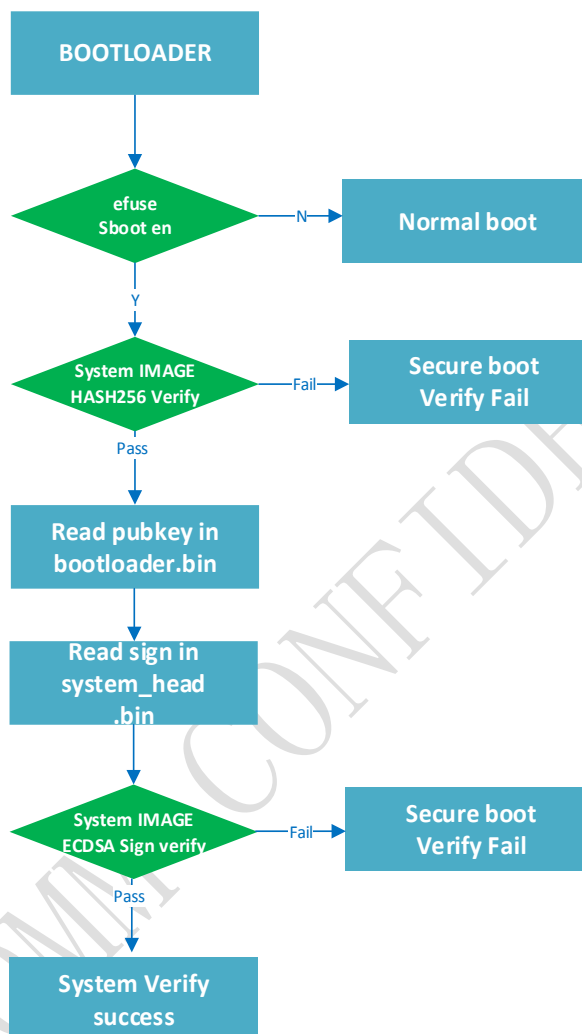
7. 芯片启动后，ROM 启动引导程序将读取 efuse 的 sboot_en, 当 sboot_en 有效时，进一步读取 efuse 的 ecdsa_puk, 并验证 bootloader image 的签名是否有效，从而验证 bootloader image 安全性。
8. 在 bootloader image 安全验证通过后，bootloader image 启动并读取嵌入在程序内部的 security bootloader pubkey, 并验证 system image 的签名是否有效，从而验证 system image 安全性。

3. 安全启动软件流程

3.1 ROM 验证 bootloader



3.2 Bootloader 验证 system



4. ECDSA 密钥及签名

安全启动方案涉及使用的是标准的 ECDSA 密钥对，包含公钥和私钥，具体参考 Image 签名算法。

验证过程需要产生两组 ECDSA 密钥对，分别用于 bootloader image 和 system image。

Security bootloader **public** key 用于 ROM 启动时验证 bootloader image 签名，

Security system public key 用于 bootloader 启动时验证 system image 签名。

4.1 密钥对的产生

生成用于 bootloader image 的密钥对文件，

ECSecTools genkey .\case_secure\securitybl_priv.pem .\case_secure\securitybl_pub.pem

生成用于 system image 的密钥对文件，

ECSecTools genkey .\case_secure\securitysys_priv.pem .\case_secure\securitysys_pub.pem

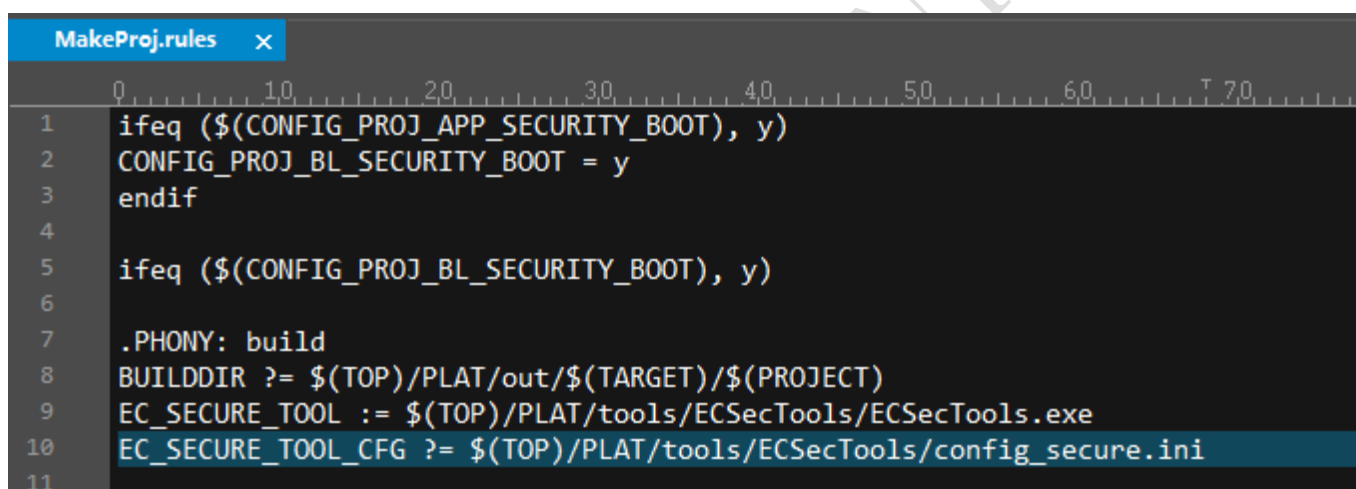
4.2 编译时使用密钥对的配置

4.2.1 编译时的配置

在 SDK Makefile 中指定 ECSecTools 工具的配置文件路径 config_secure.ini。

在 bootloader 和 app project(以 at_command 为例)的 MakeProj.rules 中设定 ECSecTools 的配置文件路径。

Bootloader 的配置:

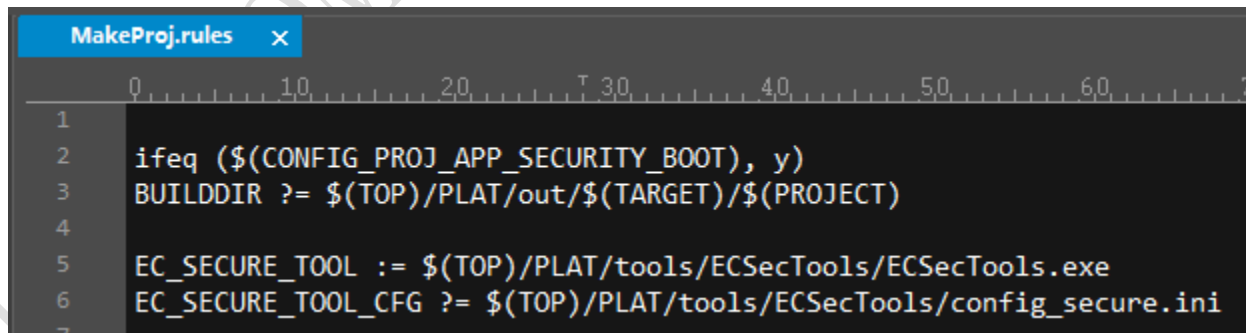


```

1  ifeq ($(CONFIG_PROJ_APP_SECURITY_BOOT), y)
2  CONFIG_PROJ_BL_SECURITY_BOOT = y
3  endif
4
5  ifeq ($(CONFIG_PROJ_BL_SECURITY_BOOT), y)
6
7  .PHONY: build
8  BUILDDIR ?= $(TOP)/PLAT/out/$(TARGET)/$(PROJECT)
9  EC_SECURE_TOOL := $(TOP)/PLAT/tools/ECSecTools/ECSecTools.exe
10 EC_SECURE_TOOL_CFG ?= $(TOP)/PLAT/tools/ECSecTools/config_secure.ini
11

```

App project 的配置:



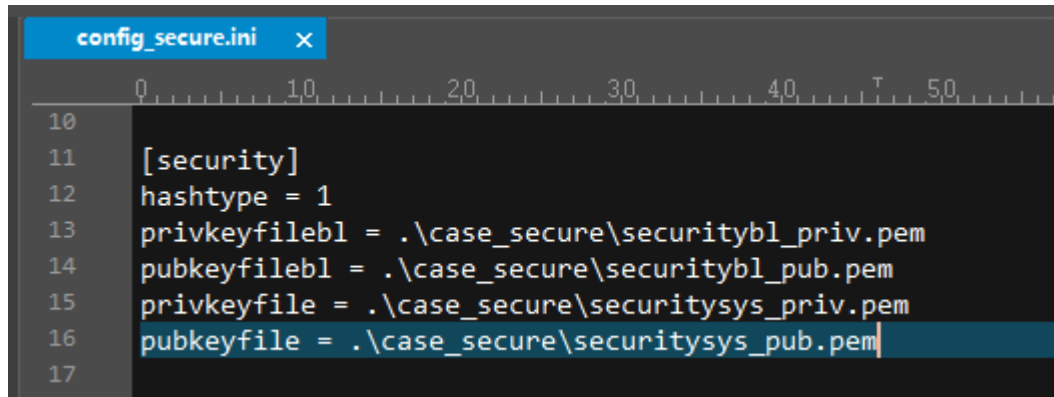
```

1
2  ifeq ($(CONFIG_PROJ_APP_SECURITY_BOOT), y)
3  BUILDDIR ?= $(TOP)/PLAT/out/$(TARGET)/$(PROJECT)
4
5  EC_SECURE_TOOL := $(TOP)/PLAT/tools/ECSecTools/ECSecTools.exe
6  EC_SECURE_TOOL_CFG ?= $(TOP)/PLAT/tools/ECSecTools/config_secure.ini
7

```

4.2.2 ECSecTools 的配置

在 config_security.ini 中配置 security section 的 privkeyfilebl/pubkeyfilebl 和 privkeyfile/pubkeyfile 分别指向之前生成的 securitybl-priv.pem/securitybl-pub.pem 和 securitysys-priv.pem/securitysys-pub.pem 文件路径。（若配置为相对路径，基础路径指向 ECSecTools 工具本身）



```

10
11 [security]
12 hashtype = 1
13 privkeyfilebl = .\case_secure\securitybl_priv.pem
14 pubkeyfilebl = .\case_secure\securitybl_pub.pem
15 privkeyfile = .\case_secure\securitysys_priv.pem
16 pubkeyfile = .\case_secure\securitysys_pub.pem
17

```

4.3 Security public key 使用

4.3.1 security bootloader public key

验证 bootloader image 的 security bootloader pubkey 需要烧录在 efuse 中，且保证是一次性烧录，从而和硬件是完全绑定的，其他用户无法不可更改。

4.3.2 security system public key

验证 system image 的 security system pubkey 需要集成编译至 bootloader image，经过安全验证的 bootloader image 是一体的，其他用户无法更改。

ECDSA security system public key 需要从 security-pub.pem 提取出来，命令如下，生成 VerifyPubkey.c:

- ECSecTools —cfgfile security_config.ini genkeysrc —output VerifyPubkey.c

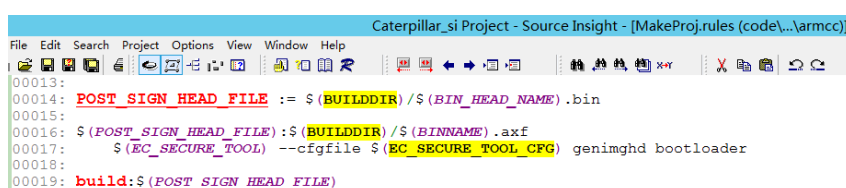
SDK 编译 bootloader 时将链接此文件至 bootloader image。

4.4 签名生成

签名保存在 bootloader_head.bin 和 app-demo-flash_head.bin 文件中。

4.4.1 Bootloader 签名

在 bootloader 和 app project 的 MakeProj.rules 配置如下，编译完成后将调用以下命令，生成 bootloader_head.bin。
ECSecTools —cfgfile config_security.ini genimgbd bootloader



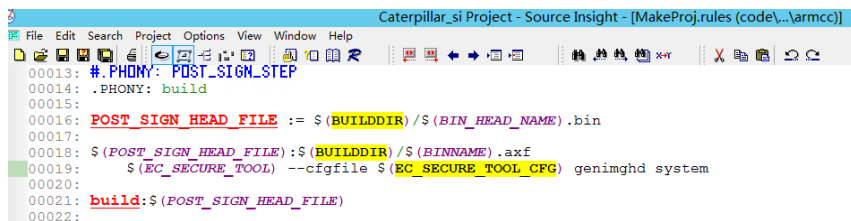
```

00013:
00014: POST_SIGN_HEAD_FILE := $(BUILDDIR)/$(BIN_HEAD_NAME).bin
00015:
00016: $(POST_SIGN_HEAD_FILE):$(BUILDDIR)/$(BINNAME).axf
00017:     $(EC_SECURE_TOOL) --cfgfile $(EC_SECURE_TOOL_CFG) genimgbd bootloader
00018:
00019: build:$(POST_SIGN_HEAD_FILE)

```

在 app project 的 MakeProj.rules 配置如下，编译完成后将调用以下命令， app-demo-flash head.bin。

```
ECSecTools -cfgfile config security.ini genimghd system
```



在 build 中依赖 ECDSA private key 对 image 进行签名，容易泄露 ECDSA private key，因此签名过程也可以放在远程服务器上进行。

远程签名时，在 SDK 上需要配置 SIGN BINARIES DURING BUILD=n，签名过程和本地签名一样。

Security bootloader/system private key 需要由用户安全保存，不能泄露，否则其他用户用该私钥进行签名的 image 都可以通过启动时的签名验证，安全启动失效。

efuse ecdsa_puk 烧录时需要同时烧录 efuse burn_dis_app（禁止烧录标志，防止反复烧录）、sboot_en(安全启动标志)。否则用户可以重新烧录 ecdsa_puk 域，导致 ecdsa_puk 失效。

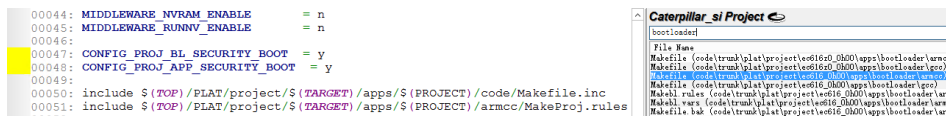
burn_dis_app 烧录之后，efuse app 域的剩余其他 bit 也无法再次烧写，因此在安全启动 efuse 烧录之前，需要提前将其他 efuse app 域的 bit 烧录完成。

Security bootloader key pair 需要妥善保存，不能遗失，否则 image 无法正常更新。安全启动启用后，相关 efuse bit 被烧录，若没有 Security bootloader key，将无法产生能通过安全验证的签名，bootloader image 烧录后无法通过安全验证。

在 SDK 中 plat\project\ec616_0h00\apps\bootload\armcc 目录下的 Makefile 中配置

CONFIG_PROJ_BL_SECURITY_BOOT=y,

```
CONFIG PROJ APP SECURITY BOOT=y
```



在 SDK 中 plat\project\ec616_0h00\apps\at_command（以 at_command 为例）目录下的 Makefile 中配置

CONFIG_PROJ_APP_SECURITY_BOOT=y

```
00012:
00013: CONFIG_PROJ_APP_SECURITY_BOOT = y
00014:
00015: include $(TOP)/PLAT/project/$(TARGET)/apps/$(PROJECT)/armcc/MakeProj.rules
00016: include $(TOP)/PLAT/tools/scripts/Makefile.rules
```

```
App.h (code/trunk/plat/project/ec616_0h00/apps/at_commandline)
App.h (code/trunk/plat/project/ec616_0h00/apps/at_commandline)
Makefile (code/trunk/plat/project/ec616_0h00/apps/at_commandline)
Makefile (code/trunk/plat/project/ec616_0h00/apps/at_commandline)
Makefile (code/trunk/plat/project/ec616_0h00/apps/at_commandline)
Bsp_custom.c (code/trunk/plat/project/ec616_0h00/apps/at_commandline)
Bsp_custom.c (code/trunk/plat/project/ec616_0h00/apps/at_commandline)
```

6. Image 签名算法

Security boot bootloader 和 app 的验证采用相同的算法-ECDSA。ECDSA 算法由 [RFC6979](#) 描述。

具体实现 ECDSA 选用的参数如下：

CURVE :secp256k1

HASH: SHA256

Key 文件格式：PEM

Bootloader 中保存的 public 采用 64raw bytes

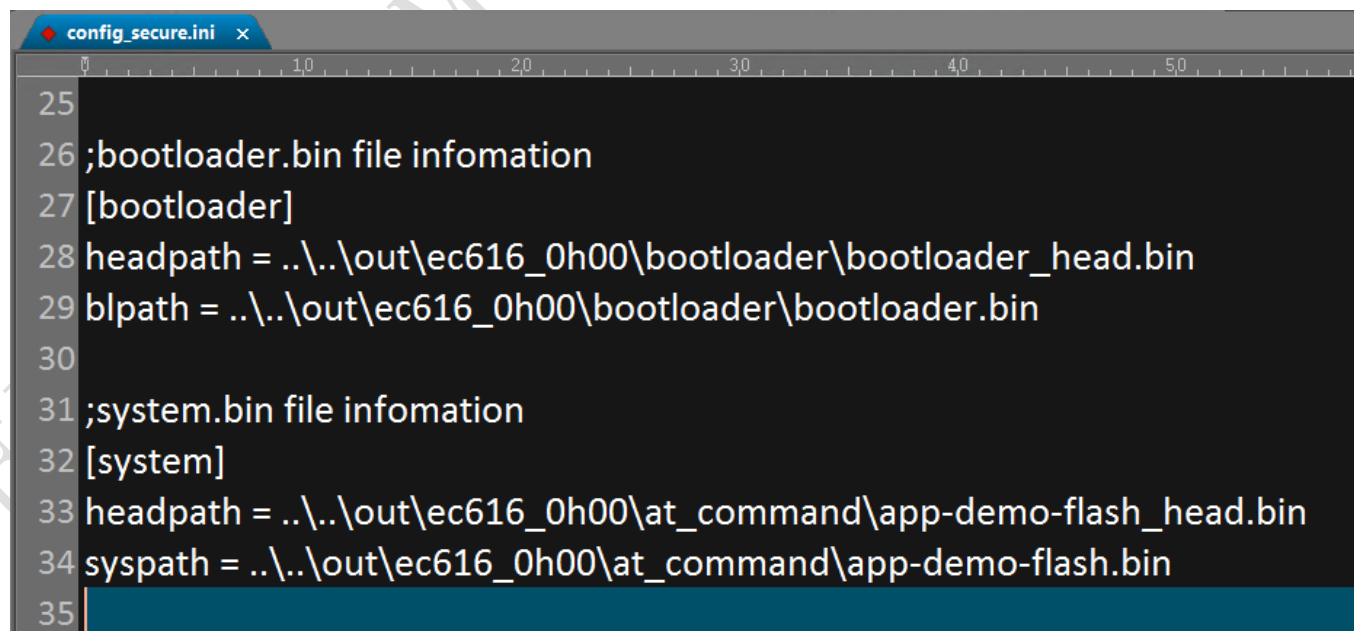
Image signature: 64 bytes

7. Image 烧录

image 烧录时需要指定 ECSecTools 配置文件 config_secure.ini,

在 config_secure.ini 中配置 bootloader section 的 headpath 为编译时生成的 bootloader_head.bin 文件所在的路径。

在 config_secure.ini 中配置 system section 的 headpath 为编译时生成的 app-demo-flash_head.bin 文件所在的路径。



```
config_secure.ini x
25
26 ;bootloader.bin file infomation
27 [bootloader]
28 headpath = ../../out/ec616_0h00/bootloader/bootloader_head.bin
29 blpath = ../../out/ec616_0h00/bootloader/bootloader.bin
30
31 ;system.bin file infomation
32 [system]
33 headpath = ../../out/ec616_0h00/at_command/app-demo-flash_head.bin
34 syspath = ../../out/ec616_0h00/at_command/app-demo-flash.bin
35
```

EiGENCOMM CONFIDENTIAL

8. 安全启动相关

启用其他安全启动相关的配置，比如禁用 JTAG，禁用 UART DOWNLOAD。

安全保存产生的 PRIVATE KEY。

使用工具的签名方法对外保密。

9. 附录

9.1 参考文档

Table 9-1. 参考文档

序号	文档名称	注释
【1】		
【2】		

9.2 术语和解释

Table 9-2. 术语和解释

术语	解释

9.3 安全和警告

EiGENCOMM CONFIDENTIAL

10. 版本

版本	日期	备注
Draft	2019-04-08	

11. 关于我们

上海移芯通信科技有限公司坐落于上海张江，公司于 2017 年 2 月成立，从事蜂窝移动通信芯片的研发和销售。公司产品主要应用于广域物联网通信领域，致力于设计最具性价比的蜂窝物联网芯片。公司团队在蜂窝通信芯片上有着辉煌历史和丰富经验。公司所有核心技术全部自研，包括算法&架构、射频、基带、SoC、协议栈软件、平台&应用软件和硬件方案等。

移芯通信首款自主研发的超低功耗 NB-IoT 芯片 EC616 于 2019 年中量产，被绝大部分头部模组企业采用，现已大批量应用于全国各区域各行业。下一代超高集成度 NB-IoT 芯片 EC616S 于 20 年底量产，超低功耗 4G Cat.1 芯片 EC618 工程样片阶段，5G 芯片正在研发中。

上海移芯通信科技有限公司

地址：中国上海市浦东新区祥科路 298 号佑越国际 1 幢 7 层

邮编：201210

电话：

电邮：

网址：<http://www.eigencomm.com>

技术支持窗口

电邮：support@eigencomm.com